SMART CONTRACT AUDIT REPORT

for

ParaSwap Augustus

Prepared By: Xiaomi Huang

PeckShield

May 26, 2024

Document Properties

Client	ParaSwap
Title	Smart Contract Audit Report
Target	Augustus
Version	1.0
Author	Xuxian Jiang
Auditors	Jason Shen, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	May 26, 2024	Xuxian Jiang	Final Release
1.0-rc1	May 5, 2024	Xuxian Jiang	Release Candidate #1

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1 Introduction

Given the opportunity to review the **Augustus** design document and related smart contract source code, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About Augustus

`ParaSwap` is a decentralized finance (`DeFi`) platform that aggregates liquidity from various decentralized exchanges (`DEXs`) to facilitate swift and efficient token swaps. It optimizes trades for the best available rates, minimizes slippage, and provides users with a user-friendly interface for seamless cryptocurrency trading across multiple blockchain networks. This audit covers the latest version of `Augustus` with several key features, including new fee model, routing, gas optimized architecture, governance time-locked admin control, as well as the `Permit2/Diamond` support. The basic information of Augustus is as follows:

Table 1.1: Basic Information of Augustus

Item	Description
Issuer	ParaSwap
Website	https://paraswap.io/
Type	Ethereum Smart Contract
Platform	Solidity
Audit Method	Whitebox
Latest Audit Report	May 26, 2024

In the following, we show the Git repository of reviewed files and the commit hash value used in this audit.

- https://github.com/paraswap/paraswap-augustus.git (0a62ec0)

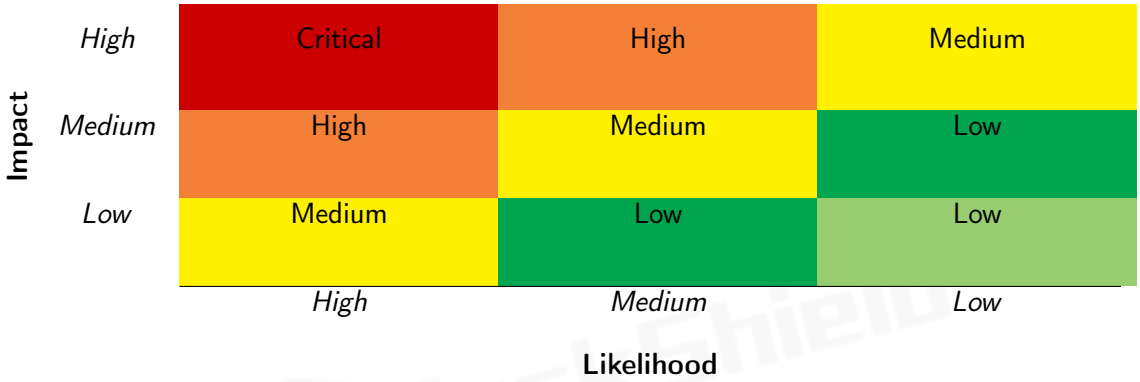
And here is the commit ID after all fixes for the issues found in the audit have been checked in:

- https://github.com/paraswap/paraswap-augustus.git (d45477e)

1.2 About PeckShield

PeckShield Inc. [9] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/peckshield), Twitter (http://twitter.com/peckshield), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification



1.3 Methodology

To standardize the evaluation, we define the following terminology based on the OWASP Risk Rating Methodology [8]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

Table 1.3: The Full Audit Checklist

Category	Checklist Items
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

To evaluate the risk, we go through a checklist of items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [7], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings. Moreover, in case there is an issue that may affect an active protocol that has been deployed, the public version of this report may omit such issue, but will be amended with full details right after the affected protocol is upgraded with respective fixes.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logic	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the implementation of the `Augustus` protocol. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logic, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	0	
Low	3	■■■
Informational	1	■
Total	4	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in Section 3.

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 3 low-severity vulnerabilities and 1 informational recommendation.

Table 2.1: Key Augustus Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Low	WETH Handling Inconsistency Among UniswapV2/V3SwapExactAmountIn	Business Logic	Resolved
PVE-002	Low	Possibly Unintended Approval in MakerPSM-RouterFacet	Business Logic	Resolved
PVE-003	Informational	Suggested Return Consistency in Fee Distribution	Coding Practices	Resolved
PVE-004	Low	Trust Issue of Admin Keys	Security Features	Resolved

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to Section 3 for details.

3 Detailed Results

3.1 WETH Handling Inconsistency Among UniswapV2/V3SwapExactAmountIn

- ID: PVE-001
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `UniswapV2/V3SwapExactAmountIn`
- Category: Business Logic [6]
- CWE subcategory: CWE-841 [3]

Description

The `Augustus` protocol aggregates liquidity from various `DEXs` to facilitate swift and efficient token swaps with improved user experience. While reviewing the aggregated support of external `DEX` engines, we notice a minor inconsistency in the `WETH`-related handling.

In the following, we show the code snippet from the related `swapExactAmountInOnUniswapV2()` routine from the `UniswapV2SwapExactAmountIn` contract. Specifically, the code snippet is about the post-swapping handling when the `destToken` is received. When `address(destToken)== address(ERC20Utils.ETH)`, it unwraps `WETH` with a smaller amount (line 90) before performing the slippage validation (line 99). For comparision, we also show the related code snippet from the `UniswapV3SwapExactAmountIn::swapExactAmountInOnUniswapV3()` counterpart. Apparently, the `UniswapV3` support performs the slippage validation before unwrapping `WETH` (and the `WETH` unwrapping amount is not decremented by 1).

```
85          // Check if destToken is ETH and unwrap
86          if (address(destToken) == address(ERC20Utils.ETH)) {
87              // Check balance of WETH
88              receivedAmount = IERC20(WETH).getBalance(address(this));
89              // Unwrap WETH
90              WETH.withdraw(receivedAmount - 1);
91              // Set receivedAmount to this contract's balance
92              receivedAmount = address(this).balance;
93          } else {
94              // Otherwise check balance of destToken
```

```
95              receivedAmount = destToken.getBalance(address(this));
96          }
97
98          // Check if swap succeeded
99          if (receivedAmount < minAmountOut) {
100             revert InsufficientReturnAmount();
101         }
```

Listing 3.1: `UniswapV2SwapExactAmountIn::swapExactAmountInOnUniswapV2()`

```
83          receivedAmount = _callUniswapV3PoolsSwapExactAmountIn(amountIn.toInt256(), pools
                , fromAddress, permit);
84
85          // Check if swap succeeded
86          if (receivedAmount < minAmountOut) {
87              revert InsufficientReturnAmount();
88          }
89
90          // Check if destToken is ETH and unwrap
91          if (address(destToken) == address(ERC20Utils.ETH)) {
92              // Unwrap WETH
93              WETH.withdraw(receivedAmount);
94          }
```

Listing 3.2: `UniswapV3SwapExactAmountIn::swapExactAmountInOnUniswapV3()`

Note a similar inconsistency also occurs in `AugustusRFQRouter::swapOnAugustusRFQTryBatchFill()`.

Recommendation Revisit the above-mentioned routines for improved post-swap consistency when handling `WETH`.

Status This issue has been resolved. The reason why the `UniswapV3` version does not decrement by 1 is because the return value is retured from `UniswapV3` calls instead of reading it from `Augustus` balance (meaning there is no 1 wei on it to be deducted)

3.2 Possibly Unintended Approval in MakerPSMRouterFacet

- ID: PVE-002
- Severity: Low
- Likelihood: Low
- Impact: Medium
- Target: `MakerPSMRouterFacet`
- Category: Business Logic [6]
- CWE subcategory: CWE-841 [3]

Description

As mentioned earlier, `Augustus` aggregates liquidity from various `DEXs`. When examining the support with the `MakerPSM` module, we notice an unintended (minor) issue that may expose leftover assets, if

any, in the Augustus router contract to third-party accounts. From another perspective, the protocol by design is not supposed to hold any asset on the router contract.

To elaborate, we show below the code snippet from the related swapExactAmountInOutOnMakerPSM() routine. Specifically, it approves the external entity, i.e., exchange or gemJoinAddress, to transfer funds on behalf of the Augustus router contract. And both exchange and gemJoinAddress addresses are directly provided by the calling user. Note the same issue is also present in other contracts, including CurveV1SwapExactAmountIn and CurveV2SwapExactAmountIn.

```
101         // Check if approve is needed
102         if (approve) {
103             if (address(srcToken) == DAI_MAKER) {
104                 // Approve exchange
105                 srcToken.approve(exchange);
106             } else {
107                 // Approve gemJoinAddress
108                 srcToken.approve(gemJoinAddress);
109             }
110         }
```

Listing 3.3: MakerPSMRouterFacet::swapExactAmountInOutOnMakerPSM()

Recommendation Validate the given user input to ensure they do not gain unauthorized approval.

Status This issue has been resolved as the protocol by design is not supposed to hold any asset on the affected router contract.

3.3 Suggested Return Consistency in Fee Distribution

- ID: PVE-003
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: AugustusFees
- Category: Coding Practices [5]
- CWE subcategory: CWE-1041 [1]

Description

The Augustus protocol has a new fee model contract AugustusFees. In the process of understanding and analyzing the new fee model, we notice a possible improvement that can be applied for better understanding and code maintenance.

Specifically, we show below the definition of two related fee distribution routines, i.e., _distributeFees() and _distributeFeesUniV3(). The former routine is used to distribute fees to the partner as well as the protocol itself while the latter is also used to distribute fees to the partner as well as the protocol

itself. Note the latter routine is used for the exclusive support of `UniswapV3`. It comes to our attention that these two routines have different return types: the former returns the new balance after the fee deduction while the latter returns the deducted fee. For consistency, we suggest to revise the latter routine to return the new balance after the fee deduction as well.

```
667     function _distributeFees(
668         uint256 currentBalance,
669         IERC20 token,
670         address payable partner,
671         uint256 partnerShare,
672         uint256 paraswapShare,
673         bool skipBlacklist,
674         bool isBlacklisted
675     )
676         private
677         returns (uint256 newBalance)
```

Listing 3.4: `AugustusFees::_distributeFees()`

```
721     function _distributeFeesUniV3(
722         uint256 currentBalance,
723         address payer,
724         IERC20 token,
725         address payable partner,
726         uint256 partnerShare,
727         uint256 paraswapShare,
728         bool skipBlacklist,
729         bool isBlacklisted
730     )
731         private
732         returns (uint256 totalFees)
```

Listing 3.5: `AugustusFees::_distributeFeesUniV3()`

Recommendation Revisit the definition of above routines so that both return the same type. Similarly, the data types of `GenericData` and `UniswapV2Data/UniswapV3Data` can be more consistent in having the same order of two same member fields.

Status This issue has been resolved as it is part of the design.

3.4 Trust Issue of Admin Keys

- ID: PVE-004
- Severity: Low
- Likelihood: Low
- Impact: Medium
- Target: `Multiple Contracts`
- Category: Security Features [4]
- CWE subcategory: CWE-287 [2]

Description

In the `Augustus` protocol, there is a privileged `owner` account that plays a critical role in governing and regulating the system-wide operations (e.g., update diamond facets/selectors, manage fee wallet, and blacklist tokens). It also has the privilege to control or govern the flow of assets managed by this protocol. Our analysis shows that the privileged account needs to be scrutinized. In the following, we examine the privileged account and the related privileged accesses in current contracts.

```
32      function setFeeWallet(address payable _feeWallet) external onlyOwner {
33          // Make sure the fee wallet is not the zero address
34          if (_feeWallet == address(0)) revert InvalidWalletAddress();
35          // Set the fee wallet
36          feeWallet = _feeWallet;
37          // Emit an event
38          emit FeeWalletUpdated(_feeWallet);
39      }
40
41      /// @inheritdoc IAdmin
42      function setFeeWalletDelegate(address payable _feeWalletDelegate) external onlyOwner
            {
43          // Make sure the fee wallet is not the zero address
44          if (_feeWalletDelegate == address(0)) revert InvalidWalletAddress();
45          // Set the fee wallet
46          feeWalletDelegate = _feeWalletDelegate;
47          // Emit an event
48          emit FeeWalletDelegateUpdated(_feeWalletDelegate);
49      }
50
51      /// @inheritdoc IAdmin
52      function setTokenBlacklisting(IERC20 token, bool isBlacklisted) public onlyOwner {
53          // Set the blacklisting status
54          blacklistedTokens[token] = isBlacklisted;
55          // Emit an event
56          emit TokenBlacklistUpdated(token, isBlacklisted);
57      }
58
59      /// @inheritdoc IAdmin
60      function batchSetTokenBlacklisting(IERC20[] calldata tokens, bool isBlacklisted)
            external onlyOwner {
61          // Loop through the tokens
```

```
62          for (uint256 i = 0; i < tokens.length; i++) {
63              // Set the blacklisting status
64              setTokenBlacklisting(tokens[i], isBlacklisted);
65          }
66      }
67
68      /// @inheritdoc IAdmin
69      function setContractPauseState(bool _isPaused) external onlyOwner {
70          // Set the pause state
71          paused = _isPaused;
72          // Emit an event
73          emit ContractPauseStateUpdated(paused);
74      }
```

Listing 3.6: Example Privileged Functions in `AdminFacet`

Note that if the privileged `owner` account is a plain EOA account, this may be worrisome and pose counter-party risk to the exchange users. A multi-sig account could greatly alleviate this concern, though it is still far from perfect. Specifically, a better approach is to eliminate the administration key concern by transferring the role to a community-governed DAO. In the meantime, a timelock-based mechanism can also be considered as mitigation.

Moreover, it should be noted that current contracts may have the support of being deployed behind a proxy. And there is a need to properly manage the proxy-admin privileges as they fall in this trust issue as well.

Recommendation Promptly transfer the privileged account to the intended `DAO`-like governance contract. All changed to privileged operations may need to be mediated with necessary timelocks. Eventually, activate the normal on-chain community-based governance life-cycle and ensure the intended trustless nature and high-quality distributed governance.

Status This issue has been resolved as the built-in `AugustusGovernance` is a timelock controller that will be bound to a multisig as owner.

4 Conclusion

In this audit, we have analyzed the Augustus design and implementation. By gathering liquidity from the main decentralized exchanges together, ParaSwap is a middleware streamlining user's interactions with various DeFi services. This audit covers the latest version of Augustus with several key features, including new fee model, routing, gas optimized architecture, governance time-locked admin control, as well as the Permit2/Diamond support. The current code base is well structured and neatly organized. Those identified issues are promptly confirmed and fixed.

Meanwhile, we need to emphasize that Solidity-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

References

[1] MITRE. CWE-1041: Use of Redundant Code. https://cwe.mitre.org/data/definitions/1041.html.

[2] MITRE. CWE-287: Improper Authentication. https://cwe.mitre.org/data/definitions/287.html.

[3] MITRE. CWE-841: Improper Enforcement of Behavioral Workflow. https://cwe.mitre.org/data/definitions/841.html.

[4] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[5] MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

[6] MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/840.html.

[7] MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

[8] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

[9] PeckShield. PeckShield Inc. https://www.peckshield.com.